

Tons of Research and Notes for the UI - Elias Christmas 4/13/2026

Initial Globe Options Notes/Research:

1. Globe.gl:

- Simple 3D spinning globe library made for visualizing data between locations
- Can rotate the globe, zoom in, click countries or cities, add markers, and draw lines connecting regions
- Completely free and doesn't require an API key, makes it easy to use for our capstone.
- Main strength is showing relationships, like music spreading between places, but the default look is more technical and less polished unless you customize it.
- Oh no I just realized it looks like its web only, I'll add a third option
- <https://globe.gl/>

2. Mapbox Globe:

- More polished interactive globe built on real map data with smooth zooming, spinning, and clickable regions
- Can highlight countries, add markers, and animate movement while also customizing the visual style to match our app UI's color schemes/aesthetic, whatever that may end up being
- Requires a free API key, but the free tier caps at an insanely high amount of users so that should be totally okay for what we're doing, and even bigger than what we're doing
- Compared to Globe.gl, it looks more refined and product-like while still supporting the same kinds of interactions
- Web and mobile friendly, and there's a dedicated library called React Native Mapbox
- <https://www.mapbox.com/blog/globe-view>

3. MapLibre:

- Free, open-source interactive map library that works on both web and React Native mobile

- Can zoom around the world, click countries or regions, add markers, draw connections, and fully customize the map style
- Completely free with no required API key, and you can use free map tile providers or host your own if needed
- Very similar capabilities to Mapbox, but without usage limits, making it a safe long-term option for a capstone
- Supports both web (MapLibre GL JS) and mobile using React Native MapLibre,
- Doesn't have a built-in spinning 3D globe like Mapbox, and is less cool looking, but totally still supports clean world maps and interactive region selection
- <https://maplibre.org/>

Final Globe Options Notes/Research:

1. Mapbox – #1 Choice

- Real 3D globe that users can spin, zoom, and interact with
- Looks the most polished and “product-level” out of all options
- Matches our idea of exploring music across the world visually
- Fully customizable to match our app’s colors and UI style
- Works on both web and mobile (Full React Native support)
- Strong “wow factor” for capstone + portfolio
- Widely used in industry, so it’s a valuable skill to learn
- Used by:
 - Companies like Facebook, Shopify, and Pinterest
 - Apps like Snapchat and Instacart
 - Media and enterprise like CNN and T-Mobile
- Free tier:
 - Free level is way more than what we would realistically need for a capstone
 - Even if we released this publicly, it would take a lot to hit the limits
 - If it somehow did blow up and hit the max, we can switch to MapLibre
 - In that case, we’d just switch over from “Discovery Globe” to “Discovery Map”
- Downsides:
- Requires API key
 - Automatically generated upon account creation and API key request, I’m familiar with this
 - Slightly more setup and potential friction
- Confidence:
- I’m confident I can build this with my current skills
- I already understand UI systems, public and private APIs, and app structure
- There’s an abundance of documentation to help me along the way

- This is more challenging than the other map option, but very doable and totally worth it for the result

2. MapLibre – #2 Choice

- Completely free and open-source
- No API key or usage limits
- Works on both web and mobile
- Users can click, drag, and zoom like a normal map
- If you recall the recent maps made by amazing community members to track where those really cold people are, to say the least, those maps were created with this API
- Reliable and easier to manage long-term
- Downsides:
 - No true spinning 3D globe, so we'd have to rename the Discovery Globe to the Discovery Map, which is okay but significantly less cool
 - Feels more like a flat map (less immersive)
 - Less “cool factor” visually
- Confidence:
 - I can quickly and easily build this if needed with next to no friction
 - Simpler setup means faster implementation
 - Strong fallback option if time or friction becomes a factor

Suggested Plan Moving Forward:

- Start with Mapbox for best visuals and experience
- If it causes too much friction, switch to MapLibre
- If we switch:
 - Rename “Discovery Globe” to “Discovery Map”

Now That I've Started Building in VS Code:

I've started building in VS Code and have some more research, feedback, and questions after looking even deeper into how we can use Mapbox for the globe/discovery part of the app.

- Main thing I learned:
 - Mapbox has a database thing specifically built for optimization in showing data on the globe, but it does not need to be our main database and often isn't, with the way this globe is used commonly in the field. We can keep our project data in our normal backend setup Leena builds, and use Mapbox specifically as the map rendering layer for optimization. Especially since I recall a legitimate concern of Leena's is that this is a lot of data to work with for the UI layer, this is a pretty cool feature I'm hoping helps alleviate that concern even more, in addition to a background process/loading screen when needed
- What I think makes the most sense for us:
 - Use a Mapbox tileset for the globe/discovery map data
 - Use our normal 'homemade' API for the rest of the app data and the detail-heavy screens
- Why this split seems good:
 - Our datasets are probably going to be massive
 - The globe only needs lightweight map-facing info
 - The country pages and other views should still use our normal API and backend logic, only the globe should use the tileset
 - That means Mapbox is doing the geospatial rendering work, while our API still handles the full relational/detail data
- For the globe specifically, each country dot/card really only needs lightweight fields, in my mockups I've included:
 - Country name
 - Hidden song count
 - Most popular album name
 - Country code (not visible to user)
 - Latitude / longitude (not visible to user)

- Year (not visible to user, since its displayed near the slider and timeline text)
- The globe doesn't need all the full country data, because the country page will hold all of that, and putting too much on the cards will make it too much mental load for users to see all of it at once
- The interaction I'm imagining:
 - One dot on the map for each country included in our dataset
 - When the user hovers on a country, the globe card shows just the simple summary info
 - When when they actually open the country page by clicking the card or the country on the list area, the app uses our normal API to get all the deeper details
- So basically:
 - Mapbox tileset = fast globe/discovery rendering
 - Our API = full app data, detail screens, comparisons, hidden gems, etc.
- Also, Milton posted two datasets with location fields in the announcement on the 15th, and at least one of them looks super helpful for this. Since we already need country-level map data, those location-related datasets may help a lot, especially since at least one already has lat/long and country mapping cleaned up.
- Another important thing I learned:
 - We'll probably be using different Mapbox implementations depending on platform:
 - Web will use Mapbox GL JS
 - Mobile will use the native/React Native Mapbox path
 - But that does NOT mean we need separate data models. We can still use the same underlying tileset/data strategy across both, just through different frontend implementations. The end result will look so similar it'll seem identical
 - I'd much rather do this than the other globe option, but I am ready to switch over if it gives me too much friction. The other option is incredibly quicker, but I'm doing great so far with the timeline with my tasks and feel confident in this

- What I learned about tilesets:
 - A tileset is basically a map-optimized version of geospatial data that Mapbox can render efficiently, especially at scale. So instead of trying to send the entire huge raw dataset straight into the client, we can prepare a lighter map-usable version of the data and turn that into the globe layer.
- What I'd want from the tileset data:
 - stable country identifier, ideally country_code
 - country name
 - lat/long for each country point
 - whatever lightweight summary fields we want to show on hover/card view, listed above
 - year for the timeline slider feature/display
- What I'd want from our normal API:
 - full country detail data
 - full song lists
 - full hidden gems data
 - comparison screen detail data
 - anything else needed for country/comparison pages/dashboard
- To Leena:
 - If you want, I can give you my Mapbox login (or look into how to make you your own like branched off mine or something?) so you can look at the tileset side of things yourself. Heads up though: Mapbox requires card info when signing up, so the account is attached to my payment info, but I used my empty Cash App card for that. Pricing-wise, the free tier currently covers up to 50,000 monthly map loads on web and up to 25,000 monthly active users on mobile, so for capstone/demo usage we should be fine. I'm also totally available to switch over to the other option at any time, even after our graduation, if we release this and it gets too many users to keep the current globe setup. I'm equally looking into implementing things with the other globe option, so I'm prepared for that if need be
 - Main question for Leena:

- I want to reword the way I asked if that's okay! I genuinely thought you might want to do that because you're data-savvy, and I'm not trying to ask you to do more work if you don't genuinely want to. I'm totally capable of doing that myself and can, just wanted to let you choose that if that's okay.
- Would you potentially be open to preparing a Mapbox-friendly globe dataset / tileset source keyed by country code, while still keeping the full app data in your regular homemade API? If not, I can totally figure out how to do it, I just thought you might genuinely like to take a look at that and might want to do it. Either way, I think it may be worth one of us making a tileset
- If yes, I think that gives us the cleanest split between globe rendering and the rest of the app

Thinking About Stored Procedures Needed (Initial Thoughts and also Since Reviewing the Mock Sql File):

- Initial:
 - I spent some time thinking through what we'd probably need for stored procedures based on the app screens and interactions. This is my current understanding of the backend data needs in general, not just for the globe, but I may be leaving some things out and I'm definitely open to your feedback on what makes the most sense structurally
 - From the UI side, I think the stored procedures should probably map to the main screens and interactions, because each screen seems to need a different level of detail
 - For the globe/discovery view, I think we need a lightweight procedure that returns one record per country for both the globe and the list view next to it. Since we're planning to show one dot per country in the dataset, this would ideally include a stable country identifier, country name, latitude/longitude, hidden song count, most

popular album name, and year if needed. The country code, lat/long, and year would mostly be used internally rather than displayed directly in the card itself. The idea is that the globe and list view only need lightweight summary information, not the full country data

- Then I think we'd want a country screen procedure that takes a country identifier and a year, since the country page will need some kind of year selector/dropdown so the displayed data can change based on the selected year. That screen seems like it should hold all the deeper country-level information, so this one would be more complete and probably include broader summary information, stats, and whatever other full-detail content we decide belongs there. On the frontend, when the year changes, I'm planning for that to trigger a smaller loading popup while the app swaps in the new data
- For the hidden gems screen, I think we'd also want a procedure that takes a country identifier and a year, because that screen will also need a year selector/dropdown and should update its data based on the selected year. That would include the list of songs for that country/year, and for each song I think we need song name, album name, genre name(s), language(s), and whatever song/track identifier or preview field we need for playback. Basically, enough to populate both the song list and the selected-song detail card on that screen
- For comparison mode, I think we'd probably need a procedure that accepts multiple country identifiers, probably 2 to 3, plus year if needed, and returns the same full/detail-level fields used for the country page, since the comparison pages are basically mirroring the country pages but in a different layout
- Depending on how search, filtering, and the timeline are handled, I could also see us needing procedures for filter/search-driven queries, especially if users can narrow by year, genre, region, or similar criteria
- So in general, the pattern I'm imagining is:
 - one lightweight globe/list-view dataset procedure
 - one country detail procedure keyed by country and year

- one hidden gems / country songs procedure keyed by country and year
 - one comparison procedure
 - possibly one or more search/filter/timeline procedures depending on how dynamic that part gets
- This is my current attempt at translating the UI into backend needs, but if there's a better or more efficient way to structure the stored procedures, I'm very open to that
- Since I've now looked through the mock SQL/schema too:
 - It helped me understand the real backend structure better
 - It confirmed for me that the frontend should not be expecting raw table rows directly
 - It looks like the data is stored across a lot of well-organized connected tables, instead of already being grouped into the exact screen-ready information the app needs
 - So some frontend values, like hidden song count, top album, top genre(s), and comparison summaries, will probably need to be calculated first in stored procedures or API logic
 - That makes me think even more that the stored procedures should be designed around app screens/interactions, not around individual tables and I'd love team feedback